01-01-10_06_001313271_001393282_001318223_001399761.pdf

Algorithms and Data Structures (ADS) - COMP1819

Develop and optimise solutions in Python with ADS and provide complexity analysis.

SHARED DOCUMENT LINK: https://uogcloud-my.sharepoint.com/:w:/g/personal/gk9453y_gre_ac_uk/EXs6D21QBTRBvQz3WuaL1h0BGmdoi8X2sLWrN4RASJjBTQ

Group Name: **GTKR**

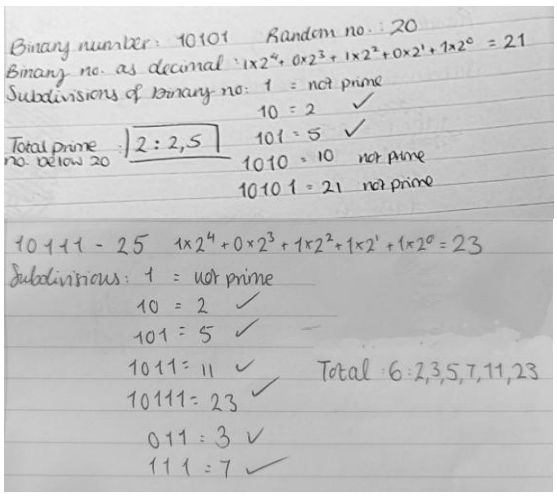
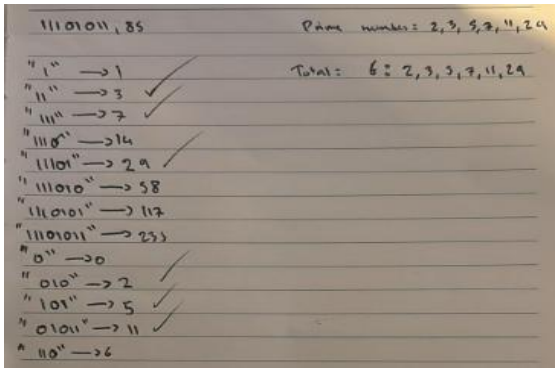
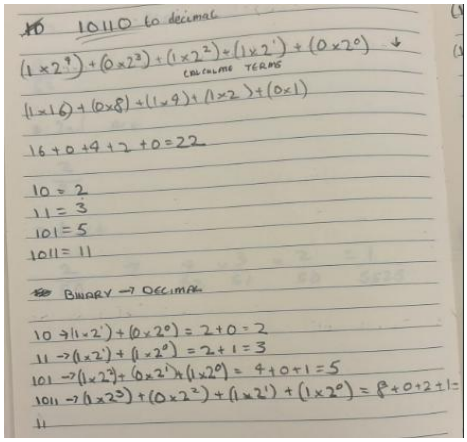
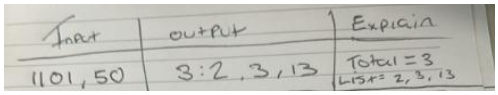
Team members: with different text colours

Member	Name	ID	Contribution %
1	Kumrili Gizem	001313271	100%
2	Andino Tonny	001393282	100%
3	Gibbs Keveroy	001318223	100%
4	Perry-Creighton Ryan	001399761	100%

Contents

1. Creating Two Basic Working Solutions

1.1 Understanding the Problem

#	Handwritten with explanation (attached)	By (student name)
1		Gizem (2 examples)
2		Tonny (1 example)
3		Keveroy (1 example)
4		Ryan (1 example)

1.2 Solution Implementations - description and highlights of the use of the data structure in your code

Solution 1:

Sieve of Eratosthenes Method: This approach marks prime numbers from binary string substrings, ensuring decimals are less than N. The Sieve of Eratosthenes is used for small N, while trial division is used for large N.

Main Features and Data Structures Used:
Sieve Generation (sieve_of_eratosthenes function):
Rationale: Precomputes primes up to N in a Boolean list.
Data Structure: The list marks indices as True (prime) or False (non-prime). Primes' multiples are flagged as False. A set comprehension converts the list to a set for faster prime lookups.
Prime Identification (find_primes_sieve function):

- List (sieve): Efficiently marks non-primes via index-based updates.
- Set (found_primes): Stores unique primes from substring processing.

Binary-to-Decimal Conversion: Utilizes efficient bitwise operation "num = (num << 1) | (1 if binary_str[j] == '1' else 0)"
Invalid Substrings: Exclude substrings starting with '0' (single '0' excluded).
Primality Test (is_prime function): Excludes numbers < 2 and even numbers > 2. vChecks odd numbers up to their square root.
Time Complexity:

- Sieve Generation: O(NloglogN).
- Prime Checking (using Set): O(1) average.
- Trial Division (large N): O(n**1/2).

Space Complexity:

- Sieve List: O(N).
- Sets (e.g., found_primes): Suitable for storage and avoiding duplicates.

Why Lists?

- Index-Based Updates: Optimal for marking non-primes.
- Sequential Memory Storage: Keeps memory usage minimal.
- Simplicity and Speed: Fast for repeatedly marking multiples.

Solution 2:
Trial Division Approach: This method identifies prime numbers from binary string substrings such that each corresponding decimal is less than N. It tests for primality by direct trial division and employs only sets for storage and lookup optimization.
Data Structure Used: Set (found_primes) - Holds unique primes, preventing duplicates by design. Offers O(1) average time for insertion and lookup, simplifying logic.
Binary-to-Decimal Conversion: Uses efficient bitwise operations "num = (num << 1) | (1 if binary_str[j] == '1' else 0)" . Evades substrings that start with '0' (except for a single '0').
Primality Test (is_prime function): Evades numbers < 2 and even numbers > 2. Checks odd numbers up to the square root for efficiency.
Time Complexity:

- Trial Division: O(n**1/2) per number.
- Substring Processing: O(L **2 · N).
- Set Operations: O(1) average.

Space Complexity: found_primes is O(P), where P is the number of distinct primes.
Why Sets?

- Uniqueness and Efficiency: No duplicates with O(1) operations.
- Simplified Logic: No duplication checks need to be done manually.
- Space Efficient: Contains only distinct primes.

1.3 Code Submission

The full source code is included in the **Appendix**

2. Testing and Comparing Solutions

2.1 Test Cases & Validation

Results for Solution 1

#	Input	Output	Pass/Fail	Running time (s)

1	Binary: 10101 Limit (N): 20	Enter the binary string: 10101 Enter the limit (N): 20 Sieve of Eratosthenes Solution Output: 2: 2, 5 Runtime: 0.00022 seconds	PASS	0.00022 seconds
2	Binary: 10111 Limit (N): 25	Enter the binary string: 10111 Enter the limit (N): 25 Sieve of Eratosthenes Solution Output: 6: 2, 3, 5, 7, 11, 23 Runtime: 0.00013 seconds	PASS	0.00013 seconds
3	Binary: 11101011 Limit (N):85	Enter the binary string: 11101011 Enter the limit (N): 85 Sieve of Eratosthenes Solution Output: 9: 2, 3, 5, 29, 43, 53 Runtime: 0.00013 seconds	PASS	0.00013 seconds
4	Binary: 10110 Limit (N):22	Enter the binary string: 10110 Enter the limit (N): 22 Sieve of Eratosthenes Solution Output: 4: 2, 3, 5, 11 Runtime: 0.00016 seconds	PASS	0.00016 seconds
5	Binary: 1101 Limit (N): 50	Enter the binary string: 1101 Enter the limit (N): 50 Sieve of Eratosthenes Solution Output: 4: 2, 3, 5, 13 Runtime: 0.00013 seconds	PASS	0.00013 seconds

Results for Solution 2

#	Input	Output	Pass/Fail	Running time (s)
1	Binary: 10101 Limit (N): 20	Enter the binary string: 10101 Enter the limit (N): 20 Trial Division Solution Output: 2: 2, 5 Runtime: 0.00013 seconds	PASS	0.00013 seconds
2	Binary: 10111 Limit (N): 25	Enter the binary string: 10111 Enter the limit (N): 25 Trial Division Solution Output: 6: 2, 3, 5, 7, 11, 23 Runtime: 0.00015 seconds	PASS	0.00015 seconds
3	Binary: 11101011 Limit (N): 85	Enter the binary string: 11101011 Enter the limit (N): 85 Trial Division Solution Output: 9: 2, 3, 5, 29, 43, 53 Runtime: 0.00015 seconds	PASS	0.00015 seconds
4	Binary: 10110 Limit (N): 22	Enter the binary string: 10110 Enter the limit (N): 22 Trial Division Solution Output: 4: 2, 3, 5, 11 Runtime: 0.00013 seconds	PASS	0.00013 seconds
5	Binary: 10110 Limit (N): 50	Enter the binary string: 1101 Enter the limit (N): 50 Trial Division Solution Output: 4: 2, 3, 5, 13 Runtime: 0.00013 seconds	PASS	0.00013 seconds

2.2 Running on Given Test Cases

Output and execution time for the 10 test cases for solution 1:

#The test case results were done via the IDE Google colab - Gizem

Test case 1: Output: 15: 2, 3, 5, 269, 2153, 17231 - Runtime: 0.14525 seconds

Test case 2: Output: 28: 2, 3, 5, 10729, 17231, 85837 - Runtime: 0.16639 seconds

Test case 3: Output: 7: 3, 7, 31, 8191, 131071, 524287 - Runtime: 0.25586 seconds
Test case 4: Output: 44: 2, 3, 5, 5571347, 41577089, 55398449 - Runtime: 0.00124 seconds
Test case 5: Output: 52: 2, 3, 5, 84087683, 3920234023, 5625434161 - Runtime: 0.00938 seconds
Test case 6: Output: 64: 2, 3, 5, 2724796139969, 5841981288761, 45810224399911 - Runtime: 0.56194 seconds
Test case 7: Output: 71: 2, 3, 5, 45810224399911, 7435453988964809, 18946016916092977 - Runtime: 12.74939 seconds
Test case 8: Output: 76: 2, 3, 5, 45810224399911, 7435453988964809, 18946016916092977 - Runtime: 17.54148 seconds
Test case 9: Output: 81: 2, 3, 5, 7435453988964809, 18946016916092977, 378518838354150661 - Runtime: 46.81164 seconds
Test case 10: Output: 89: 2, 3, 5, 7435453988964809, 18946016916092977, 378518838354150661 - Runtime: 59.71938 seconds

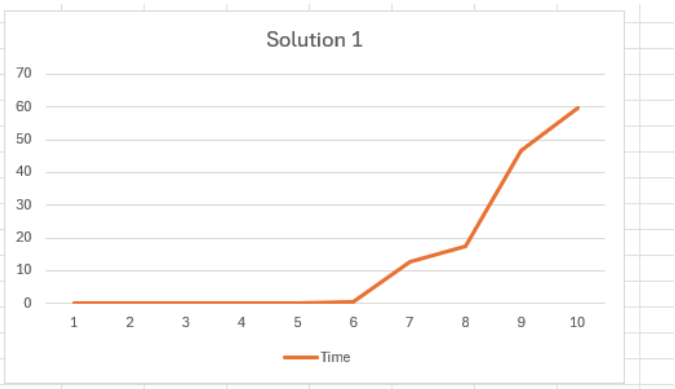
Output and execution time for the 10 test cases for solution 2:
#The test case results were done via the IDE Google colab - Gizem
Test case 1: Output: 15: 2, 3, 5, 269, 2153, 17231 - Runtime: 0.00016 seconds
Test case 2: Output: 28: 2, 3, 5, 10729, 17231, 85837 - Runtime: 0.00030 seconds
Test case 3: Output: 7: 3, 7, 31, 8191, 131071, 524287 - Runtime: 0.00050 seconds
Test case 4: Output: 44: 2, 3, 5, 5571347, 41577089, 55398449 - Runtime: 0.00120 seconds
Test case 5: Output: 52: 2, 3, 5, 84087683, 3920234023, 5625434161 - Runtime: 0.00931 seconds
Test case 6: Output: 64: 2, 3, 5, 2724796139969, 5841981288761, 45810224399911 - Runtime: 0.55001 seconds
Test case 7: Output: 71: 2, 3, 5, 45810224399911, 7435453988964809, 18946016916092977 - Runtime: 13.63687 seconds
Test case 8: Output: 76: 2, 3, 5, 45810224399911, 7435453988964809, 18946016916092977 - Runtime: 18.65293 seconds
Test case 9: Output: 81: 2, 3, 5, 7435453988964809, 18946016916092977, 378518838354150661 - Runtime: 46.79409 seconds
Test case 10: Output: 89: 2, 3, 5, 7435453988964809, 18946016916092977, 378518838354150661 - Runtime: 59.07965 seconds

2.3 Comparison of Solutions

Comparison:
Both sets of code (solution 1 and solution 2) are very similar they both use sets however solution 1 is more tailored to less complex functions when it comes to extracting unique prime numbers. This is seen in the fact that the time taken for solution 1 is faster at completing the task compared to solution 2 that takes slightly longer to complete the task it has been given this shows that solution 1 is far more lightweight than solution 2.
Solution 2 is simpler in logic compared to solution one since it does not use a sieve and relies exclusively on (is prime) to complete the entire code. However, solution 1 does not and has a (is prime) as a fallback function

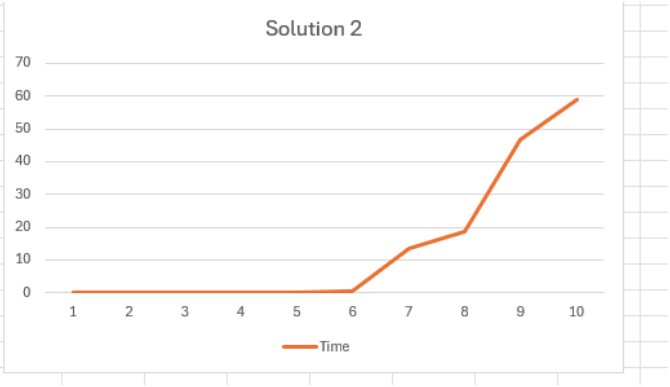
Solution 1 Graph:

Test	Time
1	0.14525
2	0.16639
3	0.25586
4	0.00124
5	0.00938
6	0.56194
7	12.74939
8	17.54148
9	46.81164
10	59.71938



Solution 2 graph:

Test	Time
1	0.00016
2	0.0003
3	0.0005
4	0.0012
5	0.00931
6	0.55001
7	13.63687
8	18.65293
9	46.79409
10	59.07965



3. Optimising Solutions

3.1 Selecting a Solution for Optimisation

We decided to optimise solution 2 since the algorithm Trial Division is less efficient than Sieve of Eratosthenes. By optimising solution 2, I aimed to improve performance and leverage more efficient number checking techniques.

3.2 Optimisation Steps & Justification

1. **Avoiding repeated primality tests (by memorising with a set)**

Original issue

The function is_prime(num) is called multiple times for the same number if it appears again in the binary substrings. This causes redundant calculations which slows down overall execution.

Optimisation?

Use a set (checked_numbers) to store numbers already checked. If a number has been tested once the function will skip checking it again.

While memorisation does introduce some overhead due to dictionary lookups, this cost is minimal compared to the significant performance gained by avoiding repeated expensive prime checks—especially for large inputs (anything larger than
106106
).

Code change

```
checked_numbers = {}  # Store prime checks

if num in checked_numbers:
    if checked_numbers[num]:
        found_primes.add(num)
    continue

if is_prime(num):
    found_primes.add(num)
    checked_numbers[num] = True
else:
    checked_numbers[num] = False
```

Performance improvements:

- Eliminates redundant is_primes() calls. (Speeding up execution time)
- Reduces unnecessary computations, making the program faster for large binary inputs.

2. **Efficient number conversion - removing if statements inside loop**

Original issue:

Branch misprediction - The conditional (if statement) inside the loop slows down performance because it needs 'branching' to make decisions.
Branching is where CPUs predicts branches (if statements) to execute code faster. This good as it speeds up computation. However, if the prediction is wrong the CPU must remove the prediction and restart which slows down the overall process. Since this if statement runs inside a loop thousand to even millions of times because its inside a loop it can make many mispredictions and those will stack up slowing down the program greatly.
Extra memory lookup - Checking 'num in checked_numbers' forces a dictionary lookup. Even though dictionary lookups are fast (O(1)), caling them repeatedly in a loop can add a lot of overhead. The second condtion 'if checked_numbers[num] forces another look up causing double the lookups.
Unnecessary processing – Every time a number appears again; the program wastes CPU cycles to verify its legitimacy instead of skipping it directly.

Optimisation?
Code change

```
if num in checked_numbers:
    if checked_numbers[num]:
        found_primes.add(num)
    continue

if num >= 2 and is_prime(num):
    found_primes.add(num)
    checked_numbers[num] = True
else:
    checked_numbers[num] = False
```

Now with this improved function the program will now only do 1 dictionary lookup halving the lookup time, avoid unnecessary if conditions for numbers already checked, and less branching which meaning smoother execution

3. **Skipping Unnecessary Substrings**

Original issue
The loop extracts all substrings, even if they aren't prime values. This wates time in unnecessary iterations.
Optimisation?

- Skip substrings starting with 0, unless it's a single digit "0"
- Use early termination for numbers go over N faster

Code change

```
if binary_str[i] == '0' and i + 1 < len(binary_str):
    continue  # Skip substrings starting with 0 unless it's the only digit.
...
if num >= N:
    break  # Stop processing once number exceeds N.
```

Performance Improvement

- Reduces the number of substrings processed.
- Cut down on unnecessary computations (increasing overall execution speeds)

4. **Improving is_prime() functions - (Improving trail division efficiency)**

Original issue
The function checks divisibility by all odd numbers up to
$\sqrt{N}$ √N
(square root n) which is inefficient as the function does too many unnecessary divisibility checks to see if the number is odd. In addition, not all odd numbers are prime so checking all these so see if they are prime is redundant.
Overall, the function spends unnecessary time checking numbers that could have been skipped, increasing execution time, especially for large 10^6
Optimisation?
Reduce modulo operations

- Instead of checking every odd number, only check divisibility by prime candidates
- Use the 6k ± 1 rule to skip unnecessary checks

Every prime number (except 2 and 3) must be of the form 6k ± 1 because every integer can be expressed as 6k, 6k+1, 6k+2, 6k+3, 6k+4, or 6k+5. Numbers of the form 6k, 6k +2, 6k +3, 6k +4 are always divisible by 2 or 3. However, only 6k +1 and 6k +5 can be prime.
So instead of checking all odd n umbers up to (square root N), we now check only those that fit the 6k ± 1 pattern. This cuts down the number of checks by almost 2/3 making the function significantly faster.

Code change

```
i = 5
while i * i <= n:
  if n % i == 0 or n % (i + 2) == 0:
    return False
  i += 6
```

5. **Using a continuous input loop**

Improvement?
Users can input multiple binary strings and N values without restarting the program which as user convenience for continuous testing and overall user efficiency.
However, a loop isn't really an optimisation.

Optimisation	**Expected performance improvement**
Memorisation of prime checks	Eliminates redundant is_prime() calls, reduces execution time.
Faster binary to decimal conversion	Remove unnecessary conditionals, improving loop efficiency
Skipping invalid substrings early	Avoids unnecessary iterations, making processing faster.
Optimised is_prime()with 6k+1 rule	Cuts down number of trial divisions, speeding up primality testing.

3.3 Code Submission

The full source code is included in the **Appendix**

4. Comparing Performance

4.1 Performance Comparison

Comparing performances

The first method uses the Sieve of Eratosthenes, a well-known algorithm with a time complexity of O(N log⁡log⁡N)O(NloglogN) that generates all prime numbers up to a given limit N. It allows for constant-time primality checks inside the precomputed range by marking non-prime values with a Boolean list (sieve). Because of this, it is quite effective in situations where frequent prime lookups are necessary. However, for very big N, particularly beyond 1 0 7 10 7, where memory consumption becomes prohibitive, its O (N) O(N) space complexity becomes a restriction. The approach uses trial division for numbers that are higher than the range of the sieve, which introduces an O(N)O(N) overhead per check and is less effective for large inputs.

The second solution, on the other hand, employs an optimised trial division method for on-demand primality verification rather than precomputing primes at all. By ignoring even integers and multiples of three and concentrating solely on numbers of the form 6k±1, it increases efficiency. To avoid doing calculations twice, it also uses memorization using a dictionary (checked numbers) that saves the outcomes of earlier primality tests. When the same number occurs more than once in various substrings of the binary input, this is especially helpful. Although the time complexity for each primality check is still O(N)O(N), this method is more memory-efficient and frequently faster for lower N N due to the use of memorization and the lack of preprocessing. The primary difference between them is found in their trade-offs: the first method performs well when there are many prime queries and a large N because quick lookups are made possible by the sieve's precomputation. However, for high N, it is impracticable and memory intensive. The second approach is more memory-efficient and does not require preprocessing overhead, which makes it more appropriate for smaller N or situations where memory is limited, even though it is less effective for repeated queries on large N. The size of N and the trade-off between performance and memory utilisation are two examples of the problem constraints that determine which option is best.

	Sieve of Eratosthenes	Trial division
purpose	Returns all primes up to a given limit	Checks if a specific number is a prime
Data structure	Boolean list and set	No data structure
Efficiency	Rapid	Slower
Time complexity	O(N log logN)	O(√N)
Space complexity	O(N) -high memory space	O(1) - constant space
Best use cases	Checking numerous small numbers repeatedly	Checking a few large numbers dynamically
limits	Uses a lot of memory	Slow for large numbers without optimization

4.2 Running on Given Test Cases

Optimised solution 2:

Test Case	Binary String:	Prime numbers	Running Time:
1	0100001101001111	15: 2, 3, 5, ..., 269, 2153, 17231	0.00009 seconds
2	01000011010011110100110101010000	28: 2, 3, 5, ..., 10729, 17231, 85837	0.00018 seconds
3	11111111111111111111111111111111111111	7: 3, 7, 31..., 8191, 131071, 524287	0.00028 seconds
4	01000011010011110100110101010000001100010011000	44: 2, 3, 5 ..., 5571347, 41577089, 55398449	0.00091 seconds
5	010000110100111101001101010100000011000100110000011 0001	52: 2, 3, 5... 84087683, 3920234023, 5625434161	0.00635 seconds
6	010000110100111101001101010100000011000100110000011 00010011001	64: 2, 3, 5..., 2724796139969, 5841981288761, 45810224399911	0.37514 seconds
7	010000110100111101001101010100000011000100110000011 00010011001000 10001	71: 2, 3, 5..., 45810224399911, 7435453988964809, 18946016916092977	8.73149 seconds
8	010000110100111101001101010100000011000100110000011 00010011001000 000101000001	76: 2, 3, 5..., 45810224399911, 7435453988964809, 18946016916092977	12.08159 seconds
9	010000110100111101001101010100000011000100110000011 00010011001000 00010100000101000100	81: 2, 3, 5..., 7435453988964809, 18946016916092977, 378518838354150661	32.11066 seconds
10	010000110100111101001101010100000011000100110000011 00010011001000 00010100000101000100010111	89: 2, 3, 5..., 7435453988964809, 18946016916092977, 378518838354150661	40.33269 seconds

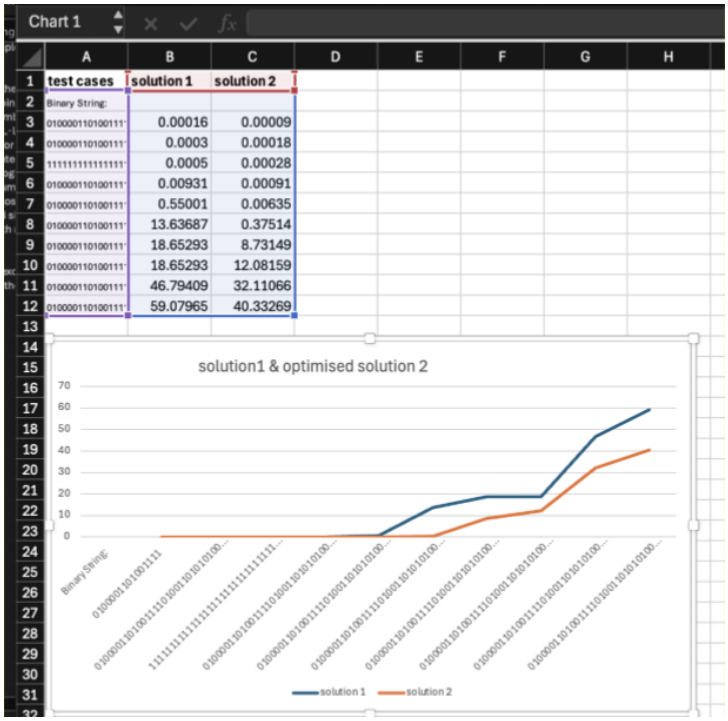
Test cases 1-10 for the optimised solution 2 proof via google colab:

```
Enter the binary string (or type 'exit' to quit): 11111111111111111111111111111111111111
Enter the limit (N): 999999

Trial Division Solution
Output: 7: 3, 7, 31, 8191, 131071, 524287
Runtime: 0.00028 seconds

Enter the binary string (or type 'exit' to quit): 0100001101001111010011010101000000110001001111000
Enter the limit (N): 999999999

Trial Division Solution
Output: 44: 2, 3, 5, 5571347, 41577089, 55398449
Runtime: 0.00091 seconds

Enter the binary string (or type 'exit' to quit): 010000110100111101001101010100000011000100111000000110001
Enter the limit (N): 123456789012

Trial Division Solution
Output: 52: 2, 3, 5, 84087683, 3920234023, 5625434161
Runtime: 0.00635 seconds

Enter the binary string (or type 'exit' to quit): 0100001101001111010011010101000000110001001110000011000100111001
Enter the limit (N): 123456789012345

Trial Division Solution
Output: 64: 2, 3, 5, 2724796139969, 5841981288761, 45810224399911
Runtime: 0.37514 seconds

Enter the binary string (or type 'exit' to quit): 0100001101001111010011010101000000110001001110000011000100111001000100001
Enter the limit (N): 123456789012345678

Trial Division Solution
Output: 71: 2, 3, 5, 45810224399911, 7435453988964809, 18946016916092977
Runtime: 8.73149 seconds

Enter the binary string (or type 'exit' to quit): 0100001101001111010011010101000000110001001110000011000100111001001000010100000001
Enter the limit (N): 1234567890123456789

Trial Division Solution
Output: 76: 2, 3, 5, 45810224399911, 7435453988964809, 18946016916092977
Runtime: 12.08159 seconds

Enter the binary string (or type 'exit' to quit): 0100001101001111010011010101000000110001001110000011000100111001001000010100000101000100
Enter the limit (N): 12345678901234567890

Trial Division Solution
Output: 81: 2, 3, 5, 7435453988964809, 18946016916092977, 378518838354150661
Runtime: 32.11066 seconds

Enter the binary string (or type 'exit' to quit): 0100001101001111010011010101000000110001001110000011000100111001001000010100000101000100010100011
Enter the limit (N): 123456789012345678901

Trial Division Solution
Output: 89: 2, 3, 5, 7435453988964809, 18946016916092977, 378518838354150661
Runtime: 40.33269 seconds

Enter the binary string (or type 'exit' to quit): exit
```

4.3 Visualising Performance

Time complexity and Big-O notations for the optimised solution: Up to a limit N, the optimal approach finds prime numbers represented by substrings of a binary text using trial division with memorization. Because it verifies each number up to √ N for primality, its temporal complexity is O(L·log⁡ N·N)O(L·logN· N), where L L is the length of the binary string. By saving findings for previously examined numbers, memorisation reduces the need for duplicate checks and increases efficiency for repeated values. O(L·log⁡N) O(L·logN) is the space complexity because it keeps the results for every distinct number that is produced. This method uses less memory than the Sieve of Eratosthenes and is more scalable for big N, but it does a little worse for little N since there was no precomputation. This makes it ideal for its scenarios with memory constricts and large inputs

Graph (done in excel) comparing runtime performance (recorded on google colab) between the optimised solution 2 and the basic solution 1



5. Reflecting on Teamwork

5.1 Contribution Marks & Team Agreement

Name	ID	Task 1 (30%)	Task 2 (20%)	Task 3 (20%)	Task 4 (15%)	Task 5 (15%)	Contribution mark (100%)
Kumrili Gizem (Group leader)	**001313271**	**30%**	**20%**	**20%**	**15%**	**15%**	**100%**
Andino Tonny	**001393282**	**30%**	**20%**	**20%**	**15%**	**15%**	**100%**
Gibbs Keveroy	**001318223**	**30%**	**20%**	**20%**	**15%**	**15%**	**100%**
Perry-Creighton Ryan	**001399761**	**10%**	**20%**	**0%**	**0%**	**0%**	**30%**

5.3 Summary of Roles & Contributions

Gizem, Developer 1 and the Group Leader was tasked with tracking deadlines and constantly giving feedback to the group so that project milestones were reached efficaciously. She also ensured report consistency and clarity while coordinating the final submission process. Her technical contribution included having pre-established solutions, ensuring that they were accurate and correct, taking runtime measurements, and keeping the team journal to have a general record of the work done by the group. Also, I helped in updating the team journal to allow tracking of the progress of the project and issues faced.

Keveroy, as Developer 2, played a vital role in refining one specific solution to achieve accuracy and efficiency. His solution was rigorously tested and became comprehensive with extensive detailed documentation. He talked constantly throughout the weeks and finished on time.

Tonny acted as the Analyst, starting the testing activity by performing the solutions with various inputs to test performance and correctness. He kept thorough records of the output and produced performance analysis by creating thought-provoking graphs and charts. His analytical knowledge was fundamental to determining the efficiency of the solution and pinpointing areas that needed possible improvement. He maintained continuity in communication throughout the weeks and completed all the work in time.

Ryan supplemented by using his analytical skills to complete task 2 and complete a handwritten sample. His communication became regular towards the end and was able to complete his work within time.

5.2 Weekly Journal & Communication Logs

Weekly journal

	Task note	Status
Week 1: 03/02/25 - 10/02		
Kumrili, Gizem (Group leader and developer)	Initiated project, communicated deadlines, and shared initial plan. Made the shared document and put all the necessary information down.	Completed
Gibbs, Keveroy (Optimiser)	Reviewed requirements and picked the task he wanted to do	Completed
Andino, Tonny (Analyst)	Reviewed requirements and picked the task he wanted to do	Completed
Perry-Creighton, Ryan (Analyst)	Reviewed requirements and picked the task he wanted to do	Completed
Week 2: 10/02 – 17/02		
Kumrili, Gizem (Group leader and developer)	Stated the handwritten task to be completed. Did 2 handwritten examples. Stated each member to solve to discuss which would be better to use and to increase contribution to the coursework.	Completed
Gibbs, Keveroy (Optimiser)	Completed handwritten example. Produced a solution.	Completed
Andino, Tonny (Analyst)	Completed handwritten example. Produced a solution.	Completed
Perry-Creighton, Ryan (Analyst)	Completed handwritten example. Helped Keveroy with his solution.	Completed
Week 3: 17/02 – 24/02		
Kumrili, Gizem (Group leader and developer)	Shared consistent updates regarding progress and assigned tasks. Wasn't getting the solutions that matched the specification therefore I decided to develop 2 solutions myself.	Completed
Gibbs, Keveroy (Optimiser)	Started optimising solution 2.	Completed
Andino, Tonny (Analyst)	Had to wait for Keveroy to finish the optimisation.	Completed
Perry-Creighton, Ryan (Analyst)	No contribution.	Pending
Week 4: 24/02 – 03/03		

Kumrili, Gizem (Group leader and developer)	Begun filling out the report and communicated updates to the group, ensuring everyone was communicating with each other.	Completed
Gibbs, Keveroy (Optimiser)	Finalised optimisation steps and confirmed solution accuracy.	Completed
Andino, Tonny (Analyst)	Started working on task 4 and analysing the solutions	Completed
Perry-Creighton, Ryan (Analyst)	No contribution.	Pending
Week 5: 03/03 – 10/03		
Kumrili, Gizem (Group leader and developer)	Oversaw everybody's progress and work and helped teammates as well as communicate.	Completed
Gibbs, Keveroy (Optimiser)	Reviewed and refined the optimised solution.	Completed
Andino, Tonny (Analyst)	Updated his analysis and refined his task.	Completed
Perry-Creighton, Ryan (Analyst)	No contribution.	Pending
Week 6: 10/03 – 17/03		
Kumrili, Gizem (Group leader and developer)	Verified all tasks were complete and updated the report. Got the input file, python file and grade report ready for uploading.	Completed
Gibbs, Keveroy (Optimiser)	Helped improve the rest of the tasks.	Completed
Andino, Tonny (Analyst)	Helped improve the rest of the tasks.	Completed
Perry-Creighton, Ryan (Analyst)	Completed task 2.	Completed

5.4 Final Report Quality

Reference

Tuan Vuong, COMP1819ADS, (2022), GitHub repository, https://github.com/vptuan/COMP1819ADS

TASK 1:

Stack Overflow. (n.d.). *efficiently finding prime numbers in python*. [online] Available at: https://stackoverflow.com/questions/46460127/efficiently-finding-prime-numbers-in-python.

boson, Z. (2014). *Sieve of Eratosthenes versus trial division time complexity*. [online] Stack Overflow. Available at: https://stackoverflow.com/questions/22858211/sieve-of-eratosthenes-versus-trial-division-time-complexity.

Wikipedia. (2020). *Sieve of Eratosthenes*. [online] Available at: https://en.wikipedia.org/wiki/Sieve_of_Eratosthenes.

OpenAI (2025). *ChatGPT*. [online] ChatGPT. Available at: https://chatgpt.com/.

Stack Overflow. (n.d.). *isPrime Function for Python Language*. [online] Available at: https://stackoverflow.com/questions/15285534/isprime-function-for-python-language.

Vishal (2018). *Python Input: Take Input from User*. [online] PYnative. Available at: https://pynative.com/python-input-function-get-user-input/.

Wikipedia. (2020). *Sieve of Eratosthenes*. [online] Available at: https://en.wikipedia.org/wiki/Sieve_of_Eratosthenes.

GeeksforGeeks (2014). *Data structures*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/data-structures/.

Python Software Foundation (n.d.). *5. Data Structures — Python 3.8.3 documentation*. [online] docs.python.org. Available at: https://docs.python.org/3/tutorial/datastructures.html.

Weisstein, E.W. (n.d.). *Prime Number*. [online] mathworld.wolfram.com. Available at: https://mathworld.wolfram.com/PrimeNumber.html.

GeeksforGeeks (2015). *Introduction to Primality Test and School Method*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/primality-test-set-1-introduction-and-school-method/

GeeksforGeeks (2024). *Sorting Algorithms - GeeksforGeeks*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/sorting-algorithms/.

www.programiz.com. (n.d.). *Python Tuple (With Examples)*. [online] Available at: https://www.programiz.com/python-programming/tuple.

docs.python.org. (n.d.). *itertools — Functions creating iterators for efficient looping — Python 3.10.0 documentation*. [online] Available at: https://docs.python.org/3/library/itertools.html#itertools.combinations.

Wikipedia Contributors (2019). *Prime number*. [online] Wikipedia. Available at: https://en.wikipedia.org/wiki/Prime_number.

nawfal (2012). *How to use actual row count (COUNT(*)) in WHERE clause without writing the same query as subquery?* [online] Stack Overflow. Available at: https://stackoverflow.com/questions/13107671/how-to-use-actual-row-count-count-in-where-clause-without-writing-the-same/34822651#34822651.

Python documentation. (n.d.). *Built-in Types*. [online] Available at: https://docs.python.org/3/library/stdtypes.html#str.isdigit.

docs.python.org. (n.d.). *8. Errors and Exceptions — Python 3.9.2 documentation*. [online] Available at: https://docs.python.org/3/tutorial/errors.html#handling-exceptions.

Python documentation. (n.d.). *5. Data Structures*. [online] Available at: https://docs.python.org/3/tutorial/datastructures.html#sets.

docs.python.org. (n.d.). *Sorting HOW TO — Python 3.10.2 documentation*. [online] Available at: https://docs.python.org/3/howto/sorting.html.

deworde (2010). *How do I refactor code into a subroutine but allow for early exit?* [online] Stack Overflow. Available at: https://stackoverflow.com/questions/3013354/how-do-i-refactor-code-into-a-subroutine-but-allow-for-early-exit/3013494#3013494.

TASK 3:

brilliant.org. (n.d.). *Sieve of Eratosthenes | Brilliant Math & Science Wiki*. [online] Available at: https://brilliant.org/wiki/sieve-of-eratosthenes/.

cp-algorithms.com. (n.d.). *Sieve of Eratosthenes - Competitive Programming Algorithms*. [online] Available at: https://cp-algorithms.com/algebra/sieve-of-eratosthenes.html.

to, B. (2020). *Binary to decimal using bitwise operators*. [online] Arduino Stack Exchange. Available at: https://arduino.stackexchange.com/questions/73272/binary-to-decimal-using-bitwise-operators

Stack Overflow. (n.d.). *Convert int to binary string in Python*. [online] Available at: https://stackoverflow.com/questions/699866/convert-int-to-binary-string-in-python.

Python, R. (n.d.). *Bitwise Operators in Python – Real Python*. [online] realpython.com. Available at: https://realpython.com/python-bitwise-operators/.

docs.python.org. (n.d.). *7. Input and Output — Python 3.8.6 documentation*. [online] Available at: https://docs.python.org/3/tutorial/inputoutput.html.

TASK 4:

GeeksforGeeks. (2017). *Sieve of Eratosthenes in 0(n) time complexity - GeeksforGeeks*. [online] Available at: https://www.geeksforgeeks.org/sieve-eratosthenes-0n-time-complexity/.

Wikipedia. (2020). *Sieve of Eratosthenes*. [online] Available at: https://en.wikipedia.org/wiki/Sieve_of_Eratosthenes.

GeeksforGeeks. (2020). *Trial division Algorithm for Prime Factorization*. [online] Available at: https://www.geeksforgeeks.org/trial-division-algorithm-for-prime-factorization/.

Free.fr. (2025). *Implementations of trial division*. [online] Available at: http://compoasso.free.fr/primelistweb/page/prime/division_en.php

Wikipedia Contributors (2019). *Big O notation*. [online] Wikipedia. Available at: https://en.wikipedia.org/wiki/Big_O_notation.

Skerritt, A. (2023). *All You Need to Know About Big O Notation [Python Examples]*. [online] Skerritt.blog. Available at: https://skerritt.blog/big-o/.

Olawanle, J. (2022). *Big O Cheat Sheet – Time Complexity Chart*. [online] freeCodeCamp.org. Available at: https://www.freecodecamp.org/news/big-o-cheat-sheet-time-complexity-chart/.

Arora, D. (2017). *Understanding Time Complexity with Simple Examples*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/understanding-time-complexity-simple-examples/.

Appendix

(Include all solutions as clear text, not screenshots)

Appendix A.1 - Proposed solution 1

```
import math

import time


def sieve_of_eratosthenes(N):
    sieve = [True] * N
    sieve[0:2] = [False, False]
    for i in range(2, int(math.sqrt(N)) + 1):
        if sieve[i]:
            for j in range(i * i, N, i):
                sieve[j] = False
    return {i for i, is_prime in enumerate(sieve) if is_prime}


def find_primes_sieve(binary_str, N):
    use_sieve = N <= 10**7
    primes = sieve_of_eratosthenes(N) if use_sieve else set()
    found_primes = set()
    max_length = len(bin(N - 1)) - 2  # Max binary length for numbers less than N

    for i in range(len(binary_str)):
        if binary_str[i] == '0' and len(binary_str) - i > 1:
            continue
        num = 0
        for j in range(i, min(i + max_length, len(binary_str))):
            num = (num << 1) | (1 if binary_str[j] == '1' else 0)
            if num >= N:
                break
            if num >= 2 and ((use_sieve and num in primes) or (not use_sieve and is_prime(num))):
                found_primes.add(num)

    primes_sorted = sorted(found_primes)
    if len(primes_sorted) < 6:
        return f"{len(primes_sorted)}: {', '.join(map(str, primes_sorted))}"
    else:
        return f"{len(primes_sorted)}: {', '.join(map(str, primes_sorted[:3] + primes_sorted[-3:]))}"
```

```
def is_prime(n):
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    for i in range(3, int(math.sqrt(n)) + 1, 2):
        if n % i == 0:
            return False
    return True


# User Input and Execution
binary_input = input("Enter the binary string: ").strip()
N = int(input("Enter the limit (N): "))


# Sieve Solution
start_time = time.time()
result_sieve = find_primes_sieve(binary_input, N)
end_time = time.time()
runtime_sieve = end_time - start_time


# Displaying Results
print("\n Sieve of Eratosthenes Solution ")
print(f"Output: {result_sieve}")
print(f"Runtime: {runtime_sieve:.5f} seconds")

#The test case results were done via the IDE Google colab - Gizem
# Test case 1: Output: 15: 2, 3, 5, 269, 2153, 17231 - Runtime: 0.14525 seconds
# Test case 2: Output: 28: 2, 3, 5, 10729, 17231, 85837 - Runtime: 0.16639 seconds
# Test case 3: Output: 7: 3, 7, 31, 8191, 131071, 524287 - Runtime: 0.25586 seconds
# Test case 4: Output: 44: 2, 3, 5, 5571347, 41577089, 55398449 - Runtime: 0.00124 seconds
# Test case 5: Output: 52: 2, 3, 5, 84087683, 3920234023, 5625434161 - Runtime: 0.00938 seconds
# Test case 6: Output: 64: 2, 3, 5, 2724796139969, 5841981288761, 45810224399911 - Runtime: 0.56194 seconds
# Test case 7: Output: 71: 2, 3, 5, 45810224399911, 7435453988964809, 18946016916092977 - Runtime: 12.74939 seconds
# Test case 8: Output: 76: 2, 3, 5, 45810224399911, 7435453988964809, 18946016916092977 - Runtime: 17.54148 seconds
# Test case 9: Output: 81: 2, 3, 5, 7435453988964809, 18946016916092977, 378518838354150661 - Runtime: 46.81164
seconds
# Test case 10: Output: 89: 2, 3, 5, 7435453988964809, 18946016916092977, 378518838354150661 - Runtime: 59.71938
seconds
```

Appendix A.2 - Proposed solution 2

```
import math

import time
```

```
def is_prime(n):

    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    for i in range(3, int(math.sqrt(n)) + 1, 2):
        if n % i == 0:
            return False
    return True

def find_primes_trial(binary_str, N):
    found_primes = set()
    max_length = len(bin(N - 1)) - 2  # Max binary length for numbers less than N

    for i in range(len(binary_str)):
        if binary_str[i] == '0' and len(binary_str) - i > 1:
            continue
        num = 0
        for j in range(i, min(i + max_length, len(binary_str))):
            num = (num << 1) | (1 if binary_str[j] == '1' else 0)
            if num >= N:
                break
            if num >= 2 and num not in found_primes and is_prime(num):
                found_primes.add(num)

    primes_sorted = sorted(found_primes)
    if len(primes_sorted) < 6:
        return f"{len(primes_sorted)}: {', '.join(map(str, primes_sorted))}"
    else:
        return f"{len(primes_sorted)}: {', '.join(map(str, primes_sorted[:3] + primes_sorted[-3:]))}"

# User Input and Execution
```

```
binary_input = input("Enter the binary string: ").strip()

N = int(input("Enter the limit (N): "))


# Trial Division Solution

start_time = time.time()

result_trial = find_primes_trial(binary_input, N)

end_time = time.time()

runtime_trial = end_time - start_time


# Displaying Results

print("\n Trial Division Solution ")

print(f"Output: {result_trial}")

print(f"Runtime: {runtime_trial:.5f} seconds")

#The test case results were done via the IDE Google colab - Gizem
# Test case 1: Output: 15: 2, 3, 5, 269, 2153, 17231 - Runtime: 0.00016 seconds
# Test case 2: Output: 28: 2, 3, 5, 10729, 17231, 85837 - Runtime: 0.00030 seconds
# Test case 3: Output: 7: 3, 7, 31, 8191, 131071, 524287 - Runtime: 0.00050 seconds
# Test case 4: Output: 44: 2, 3, 5, 5571347, 41577089, 55398449 - Runtime: 0.00120 seconds
# Test case 5: Output: 52: 2, 3, 5, 84087683, 3920234023, 5625434161 - Runtime: 0.00931 seconds
# Test case 6: Output: 64: 2, 3, 5, 2724796139969, 5841981288761, 45810224399911 - Runtime: 0.55001 seconds
# Test case 7: Output: 71: 2, 3, 5, 45810224399911, 7435453988964809, 18946016916092977 - Runtime: 13.63687 seconds
# Test case 8: Output: 76: 2, 3, 5, 45810224399911, 7435453988964809, 18946016916092977 - Runtime: 18.65293 seconds
# Test case 9: Output: 81: 2, 3, 5, 7435453988964809, 18946016916092977, 378518838354150661 - Runtime: 46.79409 seconds
# Test case 10: Output: 89: 2, 3, 5, 7435453988964809, 18946016916092977, 378518838354150661 - Runtime: 59.07965 seconds
```

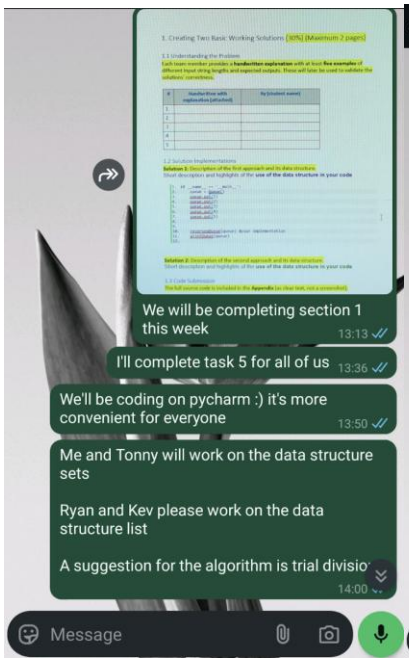
Appendix A.3 – Optimised solution

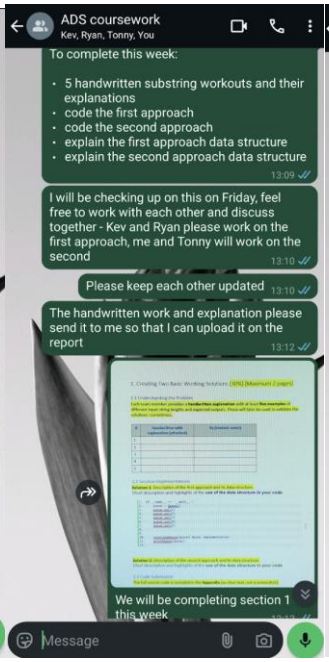
```
import math
import time

def is_prime(n):
    if n < 2:
        return False
    if n in {2, 3}:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    for i in range(5, int(math.sqrt(n)) + 1, 6):
        if n % i == 0 or n % (i + 2) == 0:
            return False
    return True


def find_primes_trial(binary_str, N):
    found_primes = set()
    checked_numbers = {}  # Memoization
    max_length = len(bin(N - 1)) - 2

    for i in range(len(binary_str)):
        if binary_str[i] == '0' and i + 1 < len(binary_str):
            continue  # Skip substrings starting with 0

        num = 0
        for j in range(i, min(i + max_length, len(binary_str))):
            num = (num << 1) | int(binary_str[j])

            if num >= N:
                break  # Stop checking numbers greater than N
```

```
            if num in checked_numbers:
                if checked_numbers[num]:
                    found_primes.add(num)
                continue

            if num >= 2 and is_prime(num):
                found_primes.add(num)
                checked_numbers[num] = True
            else:
                checked_numbers[num] = False

    primes_sorted = sorted(found_primes)
    if len(primes_sorted) < 6:
        return f"{len(primes_sorted)}: {', '.join(map(str, primes_sorted))}"
    else:
        return f"{len(primes_sorted)}: {', '.join(map(str, primes_sorted[:3] + primes_sorted[-3:]))}"


def main():
    while True:
        binary_input = input("Enter the binary string (or type 'exit' to quit): ").strip()
        if binary_input.lower() == 'exit':
            break

        N = int(input("Enter the limit (N): "))

        start_time = time.time()
        result_trial = find_primes_trial(binary_input, N)
        end_time = time.time()
        runtime_trial = end_time - start_time

        print("\nTrial Division Solution")
        print(f"Output: {result_trial}")
        print(f"Runtime: {runtime_trial:.5f} seconds\n")


if __name__ == "__main__":
    main()

#The test case results were done via the IDE Google colab FOR THE OPTIMISED SOLUTION 2 - Gizem
# Test case 1: Output: 15: 2, 3, 5, 269, 2153, 17231 - Runtime: 0.00009 seconds
# Test case 2: Output: 28: 2, 3, 5, 10729, 17231, 85837 - Runtime: 0.00018 seconds
# Test case 3: Output: 7: 3, 7, 31, 8191, 131071, 524287 - Runtime: 0.00028 seconds
# Test case 4: Output: 44: 2, 3, 5, 5571347, 41577089, 55398449 - Runtime: 0.00091 seconds
# Test case 5: Output: 52: 2, 3, 5, 84087683, 3920234023, 5625434161 - Runtime: 0.00635 seconds
# Test case 6: Output: 64: 2, 3, 5, 2724796139969, 5841981288761, 45810224399911 - Runtime: 0.37514 seconds
# Test case 7: Output: 71: 2, 3, 5, 45810224399911, 7435453988964809, 18946016916092977 - Runtime: 8.73149 seconds
# Test case 8: Output: 76: 2, 3, 5, 45810224399911, 7435453988964809, 18946016916092977 - Runtime: 12.08159 seconds
# Test case 9: Output: 81: 2, 3, 5, 7435453988964809, 18946016916092977, 378518838354150661 - Runtime: 32.11066 seconds
# Test case 10: Output: 89: 2, 3, 5, 7435453988964809, 18946016916092977, 378518838354150661 - Runtime: 40.33269 seconds
```

Appendix B - Further evidence of team contribution if necessary

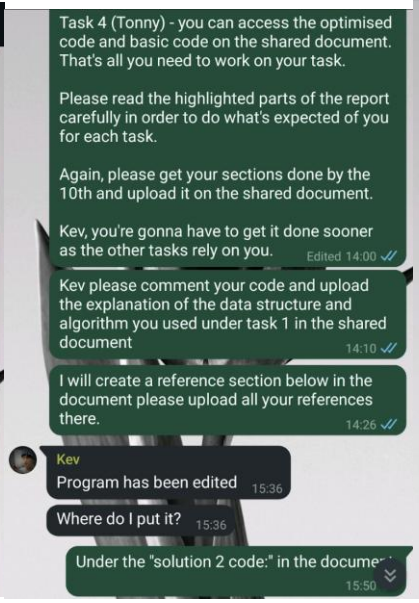
WhatsApp messages from the group chat created by Gizem (Messages are not in order by date and there were private messages between me and my teammates however I have not shared them on here due to privacy)

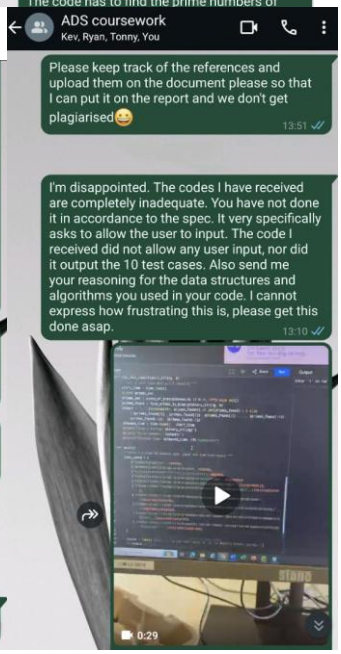

We will be completing section 1 this week 13:13
I'll complete task 5 for all of us 13:36
We'll be coding on pycharm :) it's more convenient for everyone 13:50
Me and Tonny will work on the data structure sets
Ryan and Kev please work on the data structure list
A suggestion for the algorithm is trial divisio 14:00
Message

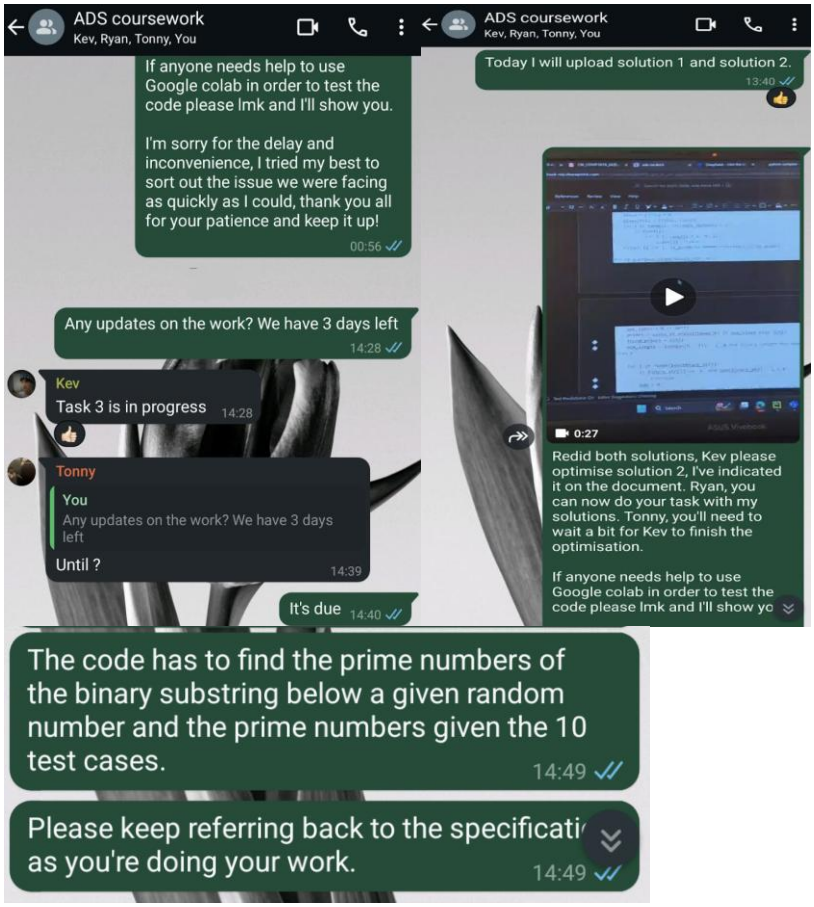

ADS coursework
Kev, Ryan, Tonny, You
To complete this week:
5 handwritten substring workouts and their explanations
code the first approach
code the second approach
explain the first approach data structure
explain the second approach data structure
13:09
I will be checking up on this on Friday, feel free to work with each other and discuss together - Kev and Ryan please work on the first approach, me and Tonny will work on the second 13:10
Please keep each other updated 13:10
The handwritten work and explanation please send it to me so that I can upload it on the report 13:12
We will be completing section 1 this week
Message


ADS coursework
Kev, Ryan, Tonny, You
Hi guys, I just want to remind everyone that our contribution points will be distributed evenly. However, in order for this to happen every member has to produce adequate work that is up to standards, if not done then your contribution points will be decreased.
Everyone must ensure that they do their own individual work which I have assigned. I would like to remind you all that our grades are given as a whole, hence we shouldn't let each other down by not doing what we are supposed to do for this coursework, this also means that no one should provide work that is done half-heartedly.
So let us all get our acts together, work efficiently and keep our communication open so that we all can achieve a grade that we can all be happy about. 14:44
Kev and Ryan, I need the first approach in code using lists and the explanation of the data structure by Sunday 23rd Feb. Please check the specification and the picture I sent of the report. 14:45
Tonny, we will be working on the second approach using sets tomorrow and explaining the data structure for it. 14:46
If any questions please ask. 14:46
Upload everything onto the word document I have created so that I can upload it onto the report. 14:46
The code has to find the prime numbers of


ADS coursework
Kev, Ryan, Tonny, You
From now on we will be distributing the coursework tasks. This requires everyone to contribute and engage with each other more. I will focus on task 1 and 5 in the report however I need the updated solution from Kev in order to do that.
Task 2 in the report - Ryan
Task 3 in the report - Kev
Task 4 in the report - Tonny
In the shared document under each picture of the tasks I have uploaded, put all your work under it so that I can put it on the actual report. I will be checking up on it often and will remind everyone. If you need any help, ask me.
All this is due on the 10th of March. We will use the rest of the week to improve on the work we have done so that I can upload it on the 18th. 13:37
Task 2 (Ryan) - you need my unoptimised code and Kev's code and the descriptions of the data structures and algorithms of them in order to complete this task. All these will be uploaded on the shared document so you can access everything you need for your task there.
Task 3 (Kev) - you will optimise solution 1 (my code) and keep solution 2 as it is. You can gain access to my code from the shared document. Please upload your code on there too.


Task 4 (Tonny) - you can access the optimised code and basic code on the shared document. That's all you need to work on your task.
Please read the highlighted parts of the report carefully in order to do what's expected of you for each task.
Again, please get your sections done by the 10th and upload it on the shared document.
Kev, you're gonna have to get it done sooner as the other tasks rely on you. Edited 14:00
Kev please comment your code and upload the explanation of the data structure and algorithm you used under task 1 in the shared document 14:10
I will create a reference section below in the document please upload all your references there. 14:26
Kev
Program has been edited 15:36
Where do I put it? 15:36
Under the "solution 2 code:" in the docume 15:50


ADS coursework
Kev, Ryan, Tonny, You
Please keep track of the references and upload them on the document please so that I can put it on the report and we don't get plagiarised 13:51
I'm disappointed. The codes I have received are completely inadequate. You have not done it in accordance to the spec. It very specifically asks to allow the user to input. The code I received did not allow any user input, nor did it output the 10 test cases. Also send me your reasoning for the data structures and algorithms you used in your code. I cannot express how frustrating this is, please get this done asap. 13:10
0:29


ADS coursework
Kev, Ryan, Tonny, You
If anyone needs help to use Google colab in order to test the code please lmk and I'll show you.
I'm sorry for the delay and inconvenience, I tried my best to sort out the issue we were facing as quickly as I could, thank you all for your patience and keep it up!
00:56
Any updates on the work? We have 3 days left
14:28
Kev
Task 3 is in progress
14:28
Tonny
You
Any updates on the work? We have 3 days left
Until ?
14:39
It's due
14:40
ADS coursework
Kev, Ryan, Tonny, You
Today I will upload solution 1 and solution 2.
13:40
0:27
Redid both solutions, Kev please optimise solution 2, I've indicated it on the document. Ryan, you can now do your task with my solutions. Tonny, you'll need to wait a bit for Kev to finish the optimisation.
If anyone needs help to use Google colab in order to test the code please lmk and I'll show yo

The code has to find the prime numbers of the binary substring below a given random number and the prime numbers given the 10 test cases.
14:49
Please keep referring back to the specificati
as you're doing your work.
14:49